

Clustering

James Burnard

11/6/2024

In this project, I will work with three different clustering techniques:

1. Hierarchical Clustering.
2. K-means Clustering.
3. Model-based Clustering.

These are the libraries I will use for this project:

```
library(NbClust)
library(mclust)

## Package 'mclust' version 6.1.1
## Type 'citation("mclust")' for citing this R package in publications.
library(flexclust)

## Loading required package: grid
## Loading required package: lattice
## Loading required package: modeltools
## Loading required package: stats4
library(cluster)
library(fpc)
library(knitr)
```

1 Data Processing.

The data set comes from the wine industry. Each row consists of 13 chemical measurements made for a specific wine plus a column “Cultivar”, which designates the type of plant used to produce the wine.

I will cluster the data using the 13 chemical measurements, ignoring Cultivar. I will then compare cluster membership with the Cultivar level to see if there is any correlation.

This analysis can help understand the relationship between cultivars and the chemistry of the wines they produce. If they are strongly related, you would see that the data forms roughly the same number of clusters as there are unique cultivars in the data set. One would expect a fairly strong correlation between cultivar and wine chemistry as the grapes are the raw material for producing the wine. If there is a weak correlation, such that one or more cultivar is represented in more than one cluster, it suggests that maybe vinters could plant cultivars that are easier to grow and that the wine would be of similar quality as compared to other cultivars that are more difficult to grow.

```
wine.data <- read.csv("wine_cultivar_data.csv")
```

After reading in the data, the next step in the analysis is to remove any missing observations.

```
print(nrow(wine.data))
```

```
## [1] 180
```

```
wine.data <- na.omit(wine.data)
print(nrow(wine.data))
```

```
## [1] 178
```

1- Make Cultivar a factor. Not assigning any labels or levels. This means the actual values will be used as the levels.

2- kable function to generate a table showing the levels of Cultivar and frequency counts.

```
wine.data$Cultivar <- factor(wine.data$Cultivar)
cultivar_freq <- as.data.frame(table(wine.data$Cultivar))
colnames(cultivar_freq) <- c("Condition", "Frequency")

knitr::kable(cultivar_freq, col.names = c("Condition", "Frequency"),
             caption = "Cultivar Levels in Dataset")
```

Table 1: Cultivar Levels in Dataset

Condition	Frequency
1	59
2	71
3	48

3 cultivars: “1”, “2”, “3”, level 2 has the highest frequency.

The next step is to prepare the data for clustering. The columns 2:14 are all numerical values, so a distance metric such as euclidean could be used; however, the range and scale of these values are very different.

This statement scales the data (excluding the column “Cultivar”) by calling the “scale” function and assigns the result to the data frame variable “wine.data.scaled”.

```
wine.data.scaled <- scale(wine.data[-1])
```

2 Part 1. Hierarchical Clustering

I will now conduct a hierarchical clustering on the scaled data set using the hclust function in the stats package (included in R). There are two basic types of hierarchical clustering algorithms, as well as several ways of judging similarity, or “linkage” between data points.

The method used by default to judge linkage is the complete linkage method. Bottom up default algorithm used by hclust, start with individual points, in each level it groups together nearby points until eventually it is one big cluster at the top of the hierarchy So for this reason it is a bottom up algorithm .

The following statement that creates a distance matrix from the scaled data set.

```
dist.mat<-dist(wine.data.scaled)
```

The following code performs a hierarchical clustering using the distance matrix and plots a dendrogram of the result.

```
clusters <- hclust(dist.mat)
plot(clusters)
```

Clusters: start: 178 , end: 1

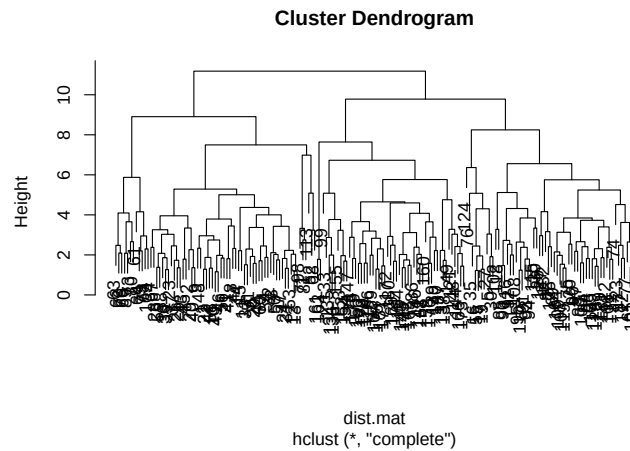


Figure 1: Hierarchical Clustering.

2.1 Choosing Number of Clusters

Now that I have a hierarchical clustering tree, I can investigate different numbers of clusters and how the number of clusters relates to the Cultivar, the type of vine.

This code chooses a cut point of 4 clusters on the tree. It also displays a table of the frequencies of the levels of Cultivar in each cluster. Finally, it generates a plot of the data points in each cluster.

```
clusterCut4 <- cutree(clusters, 4)
table(clusterCut4, wine.data$Cultivar)
```

```
##
## clusterCut4  1  2  3
##              1 51  6  0
##              2  8 50  0
##              3  0 12  0
##              4  0  3 48
```

```
plotcluster(wine.data.scaled, clusterCut4)
```

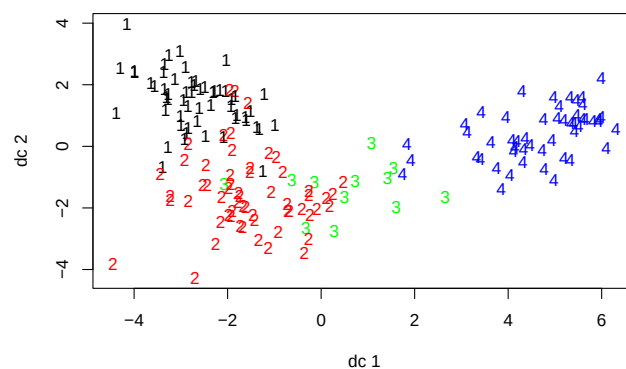


Figure 2: Hierarchical Clustering- 4 Clusters.

Based on the table, answer the following: - Cultivar 1 occurs most often in cluster 1. - No Cultivar occurs in every cluster. - Cultivar 1 & 3 occur in only one cluster

By looking at this table, it seems that Cultivar 2 occurs most frequently in cluster 2, but also 12 times in cluster 3.

Below is code that creates a cluster cut of 3, displays a table of the frequency of Cultivar for each of the three clusters, and generates a plot of the 3 clusters.

```
clusterCut4 <- cutree(clusters, 3)
table(clusterCut4, wine.data$Cultivar)
```

```
##
## clusterCut4  1  2  3
##              1 51 18  0
##              2  8 50  0
##              3  0  3 48
```

```
plotcluster(wine.data.scaled, clusterCut4)
```

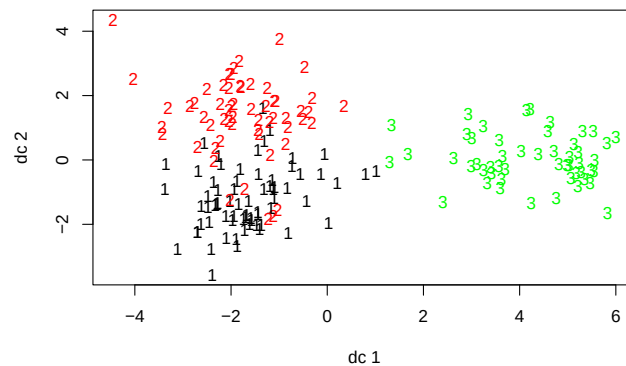


Figure 3: Hierarchical Clustering- 3 Clusters.

2.2 Cluster Purity

In the 4-cluster table, cluster 3 had the highest “purity” in terms of Cultivar because it only contained instances of Cultivar 2 and no other Cultivar. I want clusters to be as pure as possible because that means the data values in the cluster are describing something about the categorical variable of interest.

This formula provides a simple way to judge how “pure” a cluster is:

purity = max frequency/total number of data points in the cluster.

So, the purity for cluster 3 in the 4-cluster table would be: $12 / 12 = 1$, perfect purity, and the purity for cluster 2 would be: $50 / 58 = .86$

highest purity : cluster 3 , .94 , lowest purity: cluter 1 , .74

3 Part 2. Clustering with K-means

K-means is a clustering technique that is widely used. It is an example of an unsupervised learning technique. It is unsupervised because there are no pre-defined labels to serve as examples of true outcomes.

In contrast, a classifier such as KNN was an example of a supervised learning technique. It is supervised because training examples <data, label> are provided for the algorithm.

While K-means can discover clusters in data, it is up to the analyst to provide the meaning of those clusters.

The k-means algorithm is simple: given a data set and a number of clusters, k: 1- choose k cluster centers at random. 2- assign each data point to the cluster center it is closest to. 3- move the cluster center to the average point (centroid) for all of its member data points. 4- repeat steps 2 and 3 until there are no cluster reassignments or until some stopping criterion is met.

3.1 Some notable aspects of k-means.

The number of clusters has to be stated up front. Of course it makes sense to try various numbers of clusters. The results of k-means is sensitive to the initial placement of the centers (done at random). It is usual to run the algorithm many times with the same k and take the best result. The distance metric could also be changed.

3.2 Example with K-means.

The k-means algorithm is carried out by the function “kmeans”. The three parameters I will use are: the data to be clustered, the number of clusters, and the number of “runs” of the algorithm, called “nstart”.

```
kmeans(dataset, k, nstart)
```

The nstart parameter is due to the fact that the initial starting positions of the cluster centers is random and can have an effect the outcome. The value of nstart will cause kmeans to run that many different (random) starts and report the best run.

Call kmeans on the scaled wine data frame. Use 8 clusters, and nstart=25. Assign it to the variable “fit.km8”.

```
RNGversion("4.1.2")
set.seed(1234)
fit.km8 <- kmeans(wine.data.scaled, centers = 8, nstart = 25)
```

default value: 1 , starting with more initializations would mean finding better, more accurate solutions.

I can gauge the quality of the clustering in several ways: 1- The size of the clusters- are there many small clusters that could be consolidated? is there one or two big clusters? 2- The “cohesiveness” of the clusters: the distances between data points within a cluster and the cluster center. 3- The separation of clusters: how fat apart are the clusters.

```
fit.km8$size
```

```
## [1] 22 25 29 15 18 5 53 11
```

```
fit.km8$tot.withinss
```

```
## [1] 925.129
```

```
fit.km8$betweenss
```

```
## [1] 1375.871
```

These three statements print: 1- size: the size of the eight clusters. How many data points in each cluster. 2- tot.withinss: the total sum of squared distances within clusters. A measure of “cohesiveness”, the lower the better. 3- betweenss: the between cluster sum of squared distances. How well separated are the clusters, the larger the better.

Clusters:

- largest: 53
- smallest: 5
- Smaller than the between distances
- I'd say this is a good result. You'd want the distances of points within clusters to be significantly smaller than the distances of overall clusters. Based on the data given, i'd say that the k-means model I have alligns well with this statement. Of course the within stat could ideally be lower and the between stat could ideally be higher, but overall I'd say it is a good result.

As I did in Part 1, I want to see how the clustering relates to Cultivar. These statements print the contingency table for the levels of Cultivar and their frequencies in each cluster as well as a plot of the clusters.

```
table(fit.km8$cluster, wine.data$Cultivar)
```

```
##
##      1  2  3
##    1  0  1 21
##    2  0  0 25
##    3  1 28  0
##    4  5 10  0
##    5  0 18  0
##    6  0  5  0
##    7 53  0  0
##    8  0  9  2
```

```
plotcluster(wine.data.scaled, fit.km8$cluster)
```

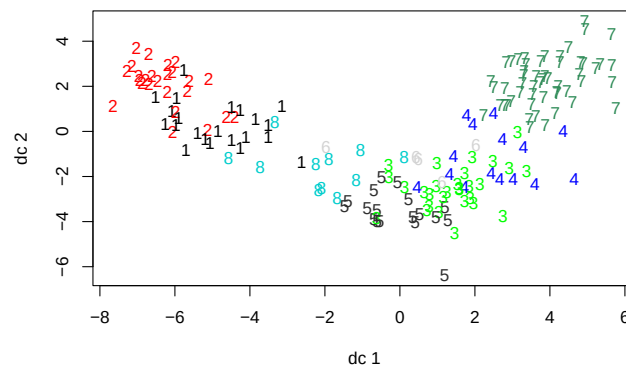


Figure 4: K-means Clustering- 8 Clusters.

Remember that the more clusters I include, the more likely I could be overfitting the data. Since I am interested in how the clusters relate to Cultivar, One way to look for this is if there is more than one cluster that has a high “purity” on the same level of Cultivar.

clusters 3, 5, and 8 have a purity above 80% for cultivar 2. I think that the number of clusters should be decreased for this reason. Also I noticed that in terms of overall purity, I believe there is one cluster with a purity below 80%. If there were a few less clusters then the purity for each cluster would increase.

3.3 Determine the optimal number of clusters.

There are many techniques for determining the optimal number of clusters in data. The measure I will be looking at: within-groups sums of squares, WSS, which measures how tightly the data points in a cluster are to each other- or to the cluster center (“cohesiveness”). From the WSS calculation, a graph is displayed showing the WSS error against the number of clusters, k. This is often called the “elbow” graph. The reason for that name will be apparent below.

A bend, or “elbow” in the graph suggests the appropriate number of clusters. Any less, and the WSS error rapidly increases.

The “elbow” graph can be produced by the following function. Note that the function uses a loop to run kmeans many times.

```
wssplot <- function(data, nc=15, seed=1234){
  wss <- (nrow(data)-1)*sum(apply(data,2,var))
  for (i in 2:nc){
    set.seed(seed)
    wss[i] <- sum(kmeans(data, centers=i)$withinss)}
}
```

```

plot(1:nc, wss, type="b", xlab="Number of Clusters",
     ylab="Within groups sum of squares")
}

```

This statement calls the `wssplot` function passing in the scaled wine data as the parameter.

```
wssplot(wine.data.scaled)
```

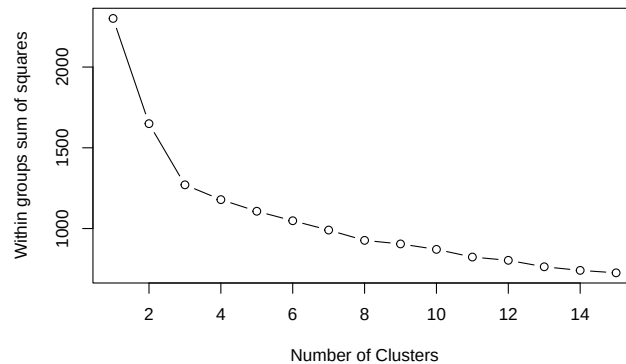


Figure 5: K-means Elbow Plot.

The “elbow” in the plot seems to be 3 clusters

The following code:

1- Calls `kmeans` on the scaled data with `k=3`, `nstart=25`, and assigns the return to the variable “`fit.km3`”. 2- Prints the total sum of squared distances within clusters (`tot.withinss`)- a measure of cluster “cohesiveness”.

```

RNGversion("4.1.2")
set.seed(1234)

fit.km3 <- kmeans(wine.data.scaled, centers = 3, nstart = 25)

fit.km3$tot.withinss

```

```
## [1] 1270.749
```

8 cluster `tot.withinss`: 1270.749 , 3 cluster `tot.withinss`: ~1200

`Withinss` decreases as the number of clusters increases. I did expect this result because the clusters keep increasing towards a the point where each data point is its own cluster. It would make sense that the distance between points within a cluster would decrease as the clusters grow smaller and smaller (which happens as a result of more clusters being added in the first place).

Code statements to display a table and plot for the clustering for `k=3`:

```

table(fit.km3$cluster, wine.data$Cultivar)

##
##      1  2  3
##  1 59  3  0
##  2  0 65  0
##  3  0  3 48

plotcluster(wine.data.scaled, fit.km3$cluster)

```

I think it is more helpful. To start, it is way more easy to read than the plot with 8 clusters. Secondly in terms of the table, I noticed that every cluster has a purity above 80%, which was not the case in for the

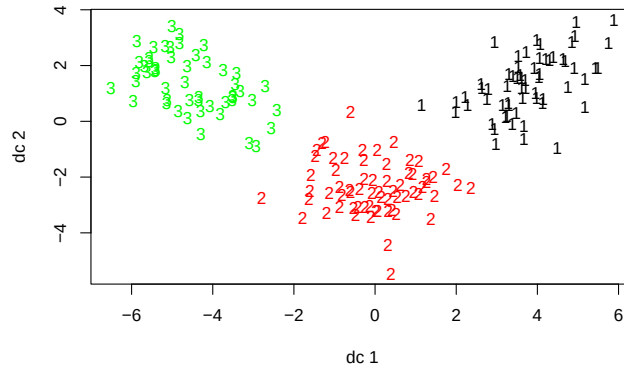


Figure 6: K-means Clustering- 3 Clusters.

model with 8 clusters. For this reason, I'd say its more efficient and a better way to evaluate the clusters.

3.4 Automated Cluster Evaluation

There are many other ways to measure the “goodness” of clustering besides WSS that can be used for judging the best number of clusters in a data set. The NbClust function in the NbClust library will use up to 30 evaluation methods (also called an index) on the data using a specific clustering method to find the optimal number of clusters.

```
nc <- NbClust(wine.data.scaled, min.nc=2, max.nc=15, method="kmeans")
```

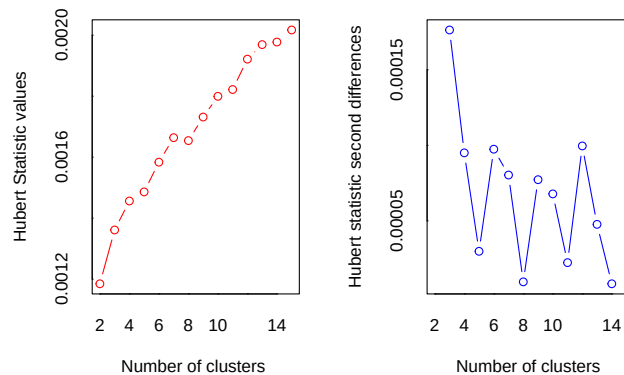


Figure 7: NbClust output.

```
## *** : The Hubert index is a graphical method of determining the number of clusters.
##           In the plot of Hubert index, we seek a significant knee that corresponds to a
##           significant increase of the value of the measure i.e the significant peak in Hubert
##           index second differences plot.
##
## *** : The D index is a graphical method of determining the number of clusters.
##           In the plot of D index, we seek a significant knee (the significant peak in Dindex
##           second differences plot) that corresponds to a significant increase of the value of
##           the measure.
##
## *****
## * Among all indices:
## * 2 proposed 2 as the best number of clusters
## * 19 proposed 3 as the best number of clusters
```

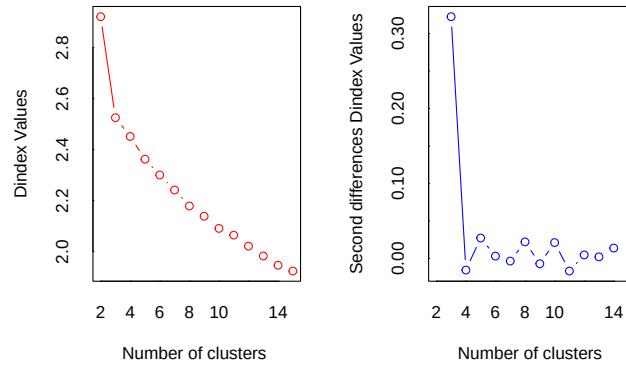



Figure 8: NbClust output.

```
## * 1 proposed 14 as the best number of clusters
## * 1 proposed 15 as the best number of clusters
##
##          ***** Conclusion *****
##
## * According to the majority rule, the best number of clusters is  3
##
##
## *****
```

```
table(nc$Best.n[1,])
```

```
##
##  0  1  2  3 14 15
##  2  1  2 19  1  1
```

The plots should correlate (roughly) with the table output. The “Dindex Values” is like the wss plot, and the “Second differences” plot shows the changes in Dindex.

19 of the evaluation methods suggested that three clusters is the best option.

3.5 Correlate Clustering with Cultivar

Another measure of similarity between two vectors of values is the Adjusted Random Index, or ARI. It ranges from -1 to 1, where 1 means the sets are the same and -1 means they are completely different. The following statement calls the “randIndex” function in the “flexclust” library to calculate ARI on the table of Cultivar and the cluster assignments for k=3.

```
randIndex(table(fit.km3$cluster, wine.data$Cultivar))
```

```
##          ARI
## 0.897495
```

ARI value: .897495 , They are very closely related.

4 Part 3. Model-based Clustering with the mclust Library

In model-based clustering, I did not specify a number of clusters. The algorithm attempts to discover the means of distinct distributions in the data. This is a more complex process than k-means.

So I’ll use the mclust library to do our model-based clustering of the wine data set.

```
wine.cluster <- Mclust(wine.data.scaled)
```

The Mclust object returned by the Mclust function call contains information about the results of the clustering.

```
summary(wine.cluster)
```

```
## -----
## Gaussian finite mixture model fitted by EM algorithm
## -----
##
## Mclust VVE (ellipsoidal, equal orientation) model with 3 components:
##
## log-likelihood  n  df      BIC      ICL
##      -2292.553 178 158 -5403.829 -5404.793
##
## Clustering table:
##   1  2  3
## 56 73 49
```

number of clusters: 3, largest: 73(cluster 2) , smallest: 49 (cluster 3)

The BIC (Bayesian Information Criterion) criterion is used to help select models with lower complexity, and to avoid overfitting the data. Models with lower BIC are generally better.

model: VVE , BIC value: -5403.829

This statement produces a graphical representation of the clustering results for all 14 models used by Mclust. -5500 is better than -7500. That is an artifact of this library.

```
plot(wine.cluster, data=wine.data.scaled, what="BIC")
```

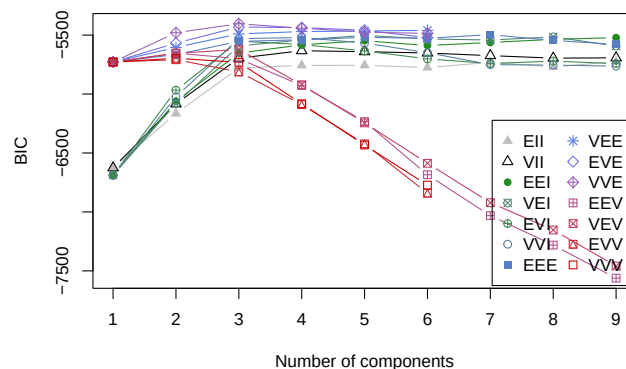


Figure 9: Model-Based BIC.

The graph reports the BIC scores for all models on the vertical axis and the number of clusters on the horizontal axis for all models.

The optimal model was reported in the summary.

4.1 Visualizing the clusters

The data has 13 dimensions, too many to graph easily. In order to visualize the clusters in 2D space, I have to find the 2 principle components (PC), of the data and use those as plotting axes.

First, I'll use the "princomp" to calculate the principle components of the data.

```
wine.pc <- princomp(wine.data.scaled, cor=T)
```

Next, I'll set up colors to represent each cluster, from 1 to 4.

```
my.color.vector <- rep("blue", times=nrow(wine.data.scaled))
my.color.vector[wine.cluster$classification==2] <- "red"
my.color.vector[wine.cluster$classification==3] <- "green"
my.color.vector[wine.cluster$classification==4] <- "orange"
```

Finally, I'll plot the clusters using the Cultivar as the label for each data point.

```
plot(wine.pc$scores[,1], wine.pc$scores[,2], ylim=range(wine.pc$scores[,1]),
     xlab="PC 1", ylab="PC 2", type='n', lwd=2)
text(wine.pc$scores[,1], wine.pc$scores[,2], labels=wine.data$Cultivar,
     cex=0.7, lwd=2, col=my.color.vector)
```

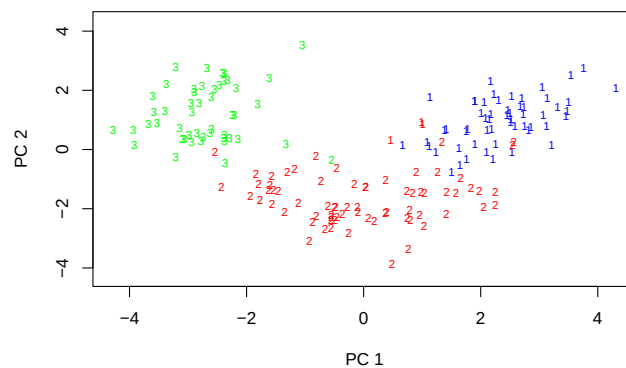


Figure 10: Model-Based Clusters.

Comparing this plot to the 3-cluster plot to those produced in Parts 1 and 2 above:

Another way to visualize the clusters:

```
#dimension reduction for visualization
model.dr <- MclustDR(wine.cluster)
plot(model.dr, what = "contour")
```

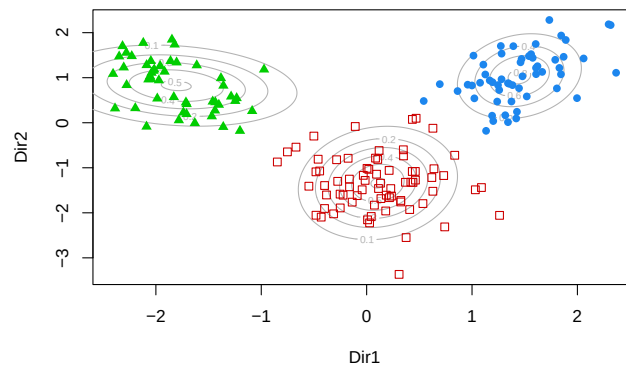


Figure 11: Model-Based Clusters.

It looks like 3 is the consensus best amount of clusters for the wine data. This makes sense considering that the within the wine data the cultivar variable has 3 levels, cultivar 1,2 & 3. Furthermore we were looking at this variable in relation to clustering, and the conclusions of the models state that three clusters is ideal. I

think that this makes perfect sense and is the right way to go about it. For each cultivar there should be some sort of cluster as cultivars play an immense role in the outcome of wine in terms of its alcohol content etc, which is just as the data set suggests.